Training Bilipschitz Embeddings of Group Quotient Spaces

Nathan Willey

Project GitHub

The goal of this project is to numerically learn maps from quotient spaces to Euclidean space $f : \mathbb{R}^d/G \to R^t$ where $G \leq O(d)$. Currently the choice of embedding dimension is $t = 3d$ for every map.

The quotient space $\mathbb{R}^d/G$ is naturally equipped with the metric which for our purposes can be written as

$$d([x],[y]) = \inf_{p\in[x],q\in[y]} \|p-q\| = \min_{g\in G} \|x-gy\|$$

Embeddings will be optimized in order to minimize worst case distortion of the quotient space. This is achieved by training small scale neural networks with various architecture whose loss function is empirical distortion of the training samples. To accomplish this, an infinite training set is assuming with density according to a standard Gaussian in $\mathbb{R}^d$, small batches of training data are sampled, and the models are trained on the empirical distortion of these batches. In this document I outline the specific procedures used to create and train such embeddings.

1 Architectures

In this section I provide the structure of each of the following embedding maps $f : \mathbb{R}^d/G \to R^t$ illustrated in Figure 1. Each map is labeled by an acronym which appears in Table 1. For $x, y \in \mathbb{R}^d$ and $G \leq O(d)$ let $\langle\langle [x],[y] \rangle\rangle = \max_{g\in G} \langle x, gy\rangle$ denote the max filter between x and y with respect to the group G. For $T \in \mathbb{R}^{t\times d}$ with rows t_i let

$$(\langle\langle [T],[x]\rangle\rangle) = (\langle\langle [t_1],[x]\rangle\rangle, \ldots, \langle\langle [t_k],[x]\rangle\rangle)^T \in \mathbb{R}^k$$

be the max filter of x with the linear map T and group G. Define sort : $\mathbb{R}^t \to \mathbb{R}^t$ to be the operator that sorts vectors in a monotonically non-increasing way. Then let $\langle\langle [x],[y]\rangle\rangle_{\text{SORT}} = \text{sort}(\langle x, y\rangle, \langle x, g_2 y\rangle \ldots, \langle x, g_k y\rangle)$ where $\{e, g_2, \ldots, g_k\}$ enumerates the elements of G. Finally for $T \in \mathbb{R}^{t\times d}$ with rows t_i let

$$\langle\langle [T],[x]\rangle\rangle_{\text{SORT}} = (\langle\langle [t_1],[x]\rangle\rangle_{\text{SORT}} \ldots \langle\langle [t_n],[x]\rangle\rangle_{\text{SORT}})^T \in \mathbb{R}^{k\cdot t}$$

Embedding Architectures:

RMF: Random Max-Filter Generate a random max-filter bank $T \in \mathbb{R}^{t\times d}$ (standard Gaussian entries).

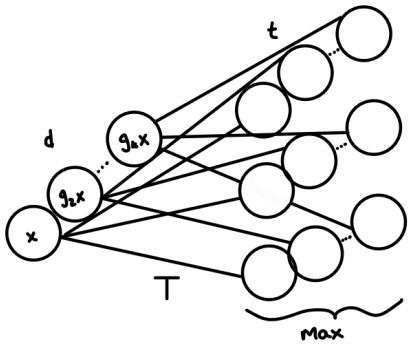
MF: Trained Max Filter Train the entries of a max-filter bank $T \in \mathbb{R}^{t \times d}$. The embedding is given by $\langle\!\langle [T], [x] \rangle\!\rangle$.

LRMF: Trained Linear Layer after Random Max-Filter Generate a random max-filter bank $T \in \mathbb{R}^{t \times d}$ (standard Gaussian entries) with rows t_i. Train a linear layer $L \in \mathbb{R}^{t \times t}$. The embedding is given by $L(\langle\!\langle [T], [x] \rangle\!\rangle)$.

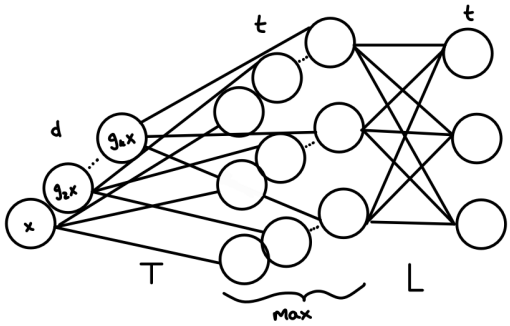
LMF: Trained Linear Layer after Trained Max Filter Train the entries of a max-filter bank $T \in \mathbb{R}^{t \times d}$ as well as a linear layer $L \in \mathbb{R}^{t \times t}$. The embedding is given by $L(\langle\!\langle [T], [x] \rangle\!\rangle)$.

SORT: Trained Sort-Based Embedding Train a template matrix $T \in \mathbb{R}^{(t/k) \times d}$. The embedding is given by $\langle\!\langle [T], [x] \rangle\!\rangle_{\text{SORT}}$.

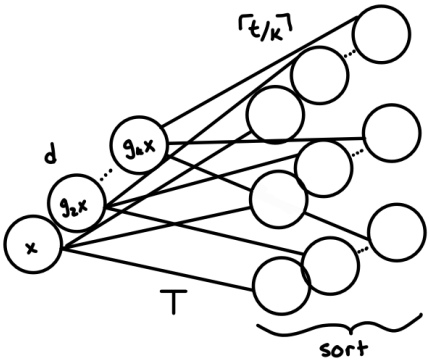
RELU: RELU Network on Orbits Train two linear layers $W \in \mathbb{R}^{kt \times d}$, $L \in \mathbb{R}^{t \times kt}$. The embedding is given by $L(\text{ReLU}(\text{Wx}))$.



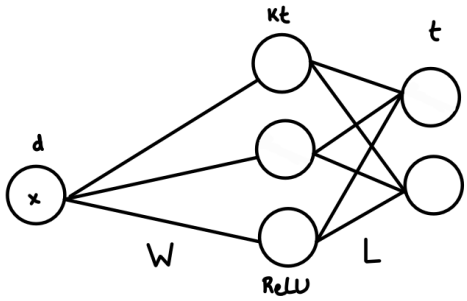
(a) Max-Filtering Embedding



(b) Linear(Max-Filtering) Embedding



(c) SORT-based embedding



(d) Single Layer ReLU Network

Figure 1: The various embedding architectures trained. The number above a layer denotes the dimensionality of that layer. A version of a) and b) with T iid Gaussian are also trained.

2 Training

Each map is trained to minimize the distortion of the embedding into its target dimension t. During training, mini-batches of data (currently $n = 20$) are randomly sampled according to the standard Gaussian and the training loss is the empirical squared worst-case distortion of the mini-batch given by $(\beta/\alpha)^2$ for measured upper Lipschitz constant β and lower Lipschitz constant α. Equivalently we can write the empirical distortion as

$$\text{loss} = \max_{i,j \in 1,\dots,n} \frac{\|f(x_i) - f(x_j)\|^2}{d([x_i],[x_j])^2} \cdot \max_{i',j' \in 1,\dots,n} \frac{d([x_{i'}],[x_{j'}])^2}{\|f(x_{i'}) - f(x_{j'})\|^2}$$

In order to compute this a quotient distance matrix for each mini-batch is computed and compared to the distance matrix resulting from the network embedding. To compute the max-filter for a **finite** group $G \leq O(d)$ of size k, each data point x is mapped to its orbit $(x, g_2x, \dots, g_kx)$. A shared linear template is then applied across the orbit, and the maximum is taken over the k transformed points. A similar process is used for the SORT embedding, but instead of taking a maximum, the outputs along the group axis are sorted and then concatenated/flattened. Since the SORT model uses the inner product of every orbit representative with the templates instead of taking the max, the number of templates is scaled down accordingly to t/k in order to end up with a final embedding dimension of t. Note that this results in the SORT embedding having fewer trainable parameters than the max-filter embedding. For the ReLU network, the input data is augmented by generating the full orbit of each point, resulting in a dataset of size kn for an original dataset of size n. Distortion is then computed on this augmented dataset. Unlike the max-filter and Sort models, the ReLU network is not inherently group-invariant. It treats each element of an orbit as a distinct input to be embedded. Because of this lack of invariance, pairs of points from the same orbit are excluded when computing the distortion loss. Their quotient distance is zero, and including them would send the loss to infinity unless the network is exactly group-invariant.

2.1 Hyperparameters

The model training process inherently comes with many tunable hyperparameters which are listed here with their current values in parentheses. The current values have not been optimized and were quickly chosen to produce something that works. For now, the hyperparameters for training every model are the identical.

- batch_size (20): Size of mini-batch of training data on which a gradient step is performed
- n_epochs (100): Number of training epochs per model. epoch determines learning rate via a scheduler. The model is tested at the end of each epoch.
- grad_steps_per_epoch (200): Number of gradient steps to take per epoch. Each gradient step is on a new mini-batch
- lr (0.01): Starting learning rate
- lr_scheduler (CosineAnnealing): Method of decreasing the learning rate over the training period

The training process is done in PyTorch using ADAM.

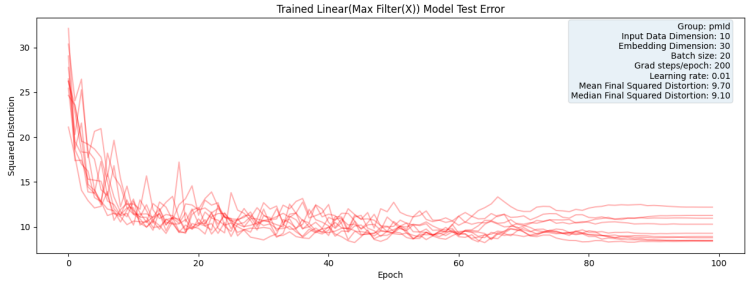


Figure 2: 10 Linear(Max-Filter) models are trained independently for embedding $\mathbb{R}^{10}/\mathbb{Z}2$ into $\mathbb{R}^{30}$. Each epoch they are evaluated on the test set and the test-set distortion is tracked and plotted.

3 Testing

Each epoch the models are evaluated on a test/validation sample. Testing is done in batches of size $\lesssim 10^4$ since, especially for ReLU whose dataset is k times larger, the distance matrices can become too large to store in memory. As of now the test sample consists of $n = 2000$ iid test points generated according to the standard Gaussian. When G is the entire symmetric group a test sample of size $n = 2000$ is infeasible even when dimensionality is as low as $d = 5$ due to the scale factor of $(d!)^2$ on the number of distances to compute for ReLU. For this example in 1 a different test set with $n = 500$ is used for ReLU. Larger test sets are possible for the other models; the bottleneck is the amount of time it takes to evaluate the loss of the ReLU model when both n and k are large. At the end of each epoch the maximum distortion is computed on the test set and recorded. An example plot of test-set distortion over the training period is shown in Figure 2. When considering the map given by random choice of a max-filter bank there is no training, instead a number of trials are run with random templates each time and their median test-set distortion is recorded. After training n_trials independent copies of a model their median test-set distortion is logged.

The final target product is a table whose columns are labeled by a choice of input dimension d, group action $G \leq O(d)$, and embedding dimension t (currently $t = 3d$ always). The columns of the table are labeled by the architecture of the embedding. The goal of this table is to compare the empirical distortions obtained by training each type of model to embed a variety of quotient spaces. Table 1 is a preliminary version of this table.

	$\mathbb{R}^3/\mathbb{Z}2 \to \mathbb{R}^9$	$\mathbb{R}^{10}/\mathbb{Z}2 \to \mathbb{R}^{30}$	$\mathbb{R}^5/S_d \to \mathbb{R}^{15}$	$\mathbb{R}^{20}/C_d \to \mathbb{R}^{60}$	$\mathbb{R}^{50}/C_d \to \mathbb{R}^{150}$
RMF	25.73	21.89	93.58	27.50	14.49
MF	3.44	8.19	10.53	16.65	10.22
SORT	13.01	24.05	1.00	29.01	29.2
LRMF	19.47	16.41	1.00	6.96	4.31
LMF	3.25	9.10	1.00	5.94	4.14
ReLU	3.02	9.34	1,256	16.42	4.49

Table 1: Final test-set distortions for each trained embedding on various quotient spaces using the parameters detailed in Section 2.1. $\mathbb{Z}2$ acts by a diagonal sign change. S_d acts by standard permutations of vector entries. C_d acts by cyclically translating vector entries.